Contents

SPECIFICATION REVIEW REPORT — Minimal Coding Agent (chapter 15)

Target: `SPEC.md` — *Minimal Coding Agent (closed-loop control, context engineering, tool use, permission gate, verification, trajectory instrumentation, + uv)*, status **v0.1 Reviewer:** `spec-review` skill (4-pass method: comprehension → local precision → cross-consistency → implementation simulation) **Grounding:** pure spec-quality review — no `src/` or `tests/` exist yet for this lab, so all findings are *spec defects* (not spec<->code divergence). Each finding is checked against the skill's core test: *"What would a competent implementer still have to guess?"* **Maturity scale:** 0 = absent · 1 = seriously deficient · 2 = weak · 3 = adequate · 4 = strong · 5 = implementation-grade

---

1. Executive Summary

`SPEC.md` v0.1 is a **strong Level 2+** specification for chapter 15's *Build a Minimal Coding Agent* exercise. It is unusually disciplined for a lab deliverable: it decomposes the chapter into a clean deterministic-vs-probabilistic seam; pins the §17 instrumentation field set; gives a closed-loop control diagram (§3.2); enumerates a full edge-case catalog (E-01…E-13); and carries week-1's `MockPolicy`/model-availability discipline forward. Every requirement traces to a contract and to a test in §11, and the invariants I-001…I-013 are each mechanically checkable.

It is **not yet Level 3** for a small but real set of defects, of which four are **HIGH** (a competent verifier cannot resolve the ambiguity without inventing a convention, and the two most dangerous are *contradictions inside the current document*):

- **F-001 (HIGH)** — the **iteration index base is mixed**: C-06/C-07 JSON use 0-based `"iteration"` / `"iteration":1/2/3`, while T-01/T-04/T-06 prose says "iteration 1/3/5" and `iterations_used == 1`. The byte-identity tests T-09/T-11 depend on a fixed convention that is not pinned.
- **F-002 (HIGH)** — the **"N consecutive `ERROR/DENY` → terminate"** rule is cross-referenced to **K-04**, which is in fact the *sandbox-size* bound, not an error/deny budget; the threshold count and its CLI flag are undefined. A verifier cannot test termination-by-stalled-errors.

- **F-003 (HIGH)** — the `ERROR` `final_outcome` **exit code is contradictory**: C-08 says `ERROR → exit 1 / 4`, K-02 and §5.1 reserve exit `4` for `PULL_REQUIRED` only, and E-02/E-03 say "exit `1/4`". Which one?
- **F-005 (HIGH)** — the **§17 fixture repository and its canonical defect are unspecified**, yet T-04/T-11 assert concrete, byte-reproducible numbers (`iterations_to_verified == 3`). Without a pinned fixture, two implementers cannot produce the same trace.

Two further **cross-consistency** defects stand out: **F-004 (MEDIUM)** — `STALLED` is *overloaded* (no-op/stuck-loop vs no-compact budget overflow) with no threshold and no defined relationship between the two; **F-006 (MEDIUM)** — `compare` is invoked in prose (§3.3, E-05, R-16/R-18 `--baseline`) but is **not** a subcommand in §5.1.

Strengths (most important).

- A clean **deterministic/probabilistic seam** with a `MockPolicy` double and a carried model-availability taxonomy (R-13/R-14, E-11/E-12, T-12) — the whole suite runs offline.
- A **closed, pinned action/tool space** (C-02/C-03, I-004) and a **closed** `final_outcome`/`Verdict` **set** (C-08, C-05).
- A **complete, measurable edge-case catalog** (E-01…E-13) with named exit codes, well beyond what most lab specs provide.
- **Permission-outside-the-model** is stated as an invariant (I-008) and tested (T-03) — the chapter's key security thesis is made mechanical.

Weaknesses (most important).

- **Conventions that byte-stability depends on are not pinned** — iteration base (F-001), the consecutive-failure budget (F-002), the `ERROR` exit code (F-003) — so T-04/T-06/T-09 cannot be written unambiguously.
- **The §17 acceptance experiment names numbers but not the fixture** (F-005) — the highest-leverage fix.
- **Two overloaded/omitted terms** (`STALLED`, `compare`) and a **module-name drift** (`loop.py` vs `control_loop.py`, F-011) that a cross-walker trips on.

Findings by severity. 0 CRITICAL · 4 HIGH (F-001, F-002, F-003, F-005) · 8 MEDIUM (F-004, F-006, F-007, F-008, F-009, F-010, F-011, F-013) · 4 LOW (F-012, F-014, F-015, F-016). Total: **16 findings**. Severities are stated per finding in §4; §20 prioritizes them P0/P1/P2.

Primary blocker: the §17 fixture + canonical defect (F-005), together with the unpinned conventions F-001/F-002/F-003 — none of which let a verifier write a *deterministic* test of the very experiment the chapter is built around.

Recommendation: resolve the 4 HIGH + 8 MEDIUM items (all P0/P1; see §20) to reach **Level 3**. Every fix is local and additive — a couple of definition sentences, a convention pin, and two cross-reference corrections — **not a re-**

design. No architectural change is recommended (§17). Per the skill, the LOW set (F-012…F-016) MAY be deferred.

---

2. Overall Maturity

Level 3 — Implementation-grade, *pending* the P0/P1 fixes in §20 (presently a strong **Level 2+**).

The specification clears most of the gate to Level 3 already: a competent coding agent can implement the deterministic harness (control loop, context manager, tool controller, permission layer, verifier, instrumenter, reporter) with minimal semantic inference, and those layers carry mechanically checkable invariants (I-001…I-013) and a full edge-case catalog (E-01…E-13). The probabilistic path (Ollama) is honestly bounded as *best-effort / opt-in*, which is the correct treatment of nondeterminism rather than a pretend-away.

It does **not** yet reach Level 3 cleanly because of one class of defect: **a convention that a conforming implementation must agree on is left to guess, or contradicts itself** (F-001 iteration base, F-002 consecutive-failure budget, F-003 `ERROR` exit code, F-005 fixture). Per the skill's core test — *could two competent implementers build materially different systems?* — two implementers working strictly from v0.1 could disagree on the iteration index, the error/deny termination threshold, *and* which fixture to inject, and each would produce a **different byte stream** for `trajectory.json`, defeating the very byte-identity invariant (I-002) the spec claims. A verifier working from v0.1 could not *write* T-04 (`iterations_to_verified == 3`) or T-06 (stop at "iteration 5") without first inventing a base.

After the P0/P1 set is integrated (pin the iteration base, define the consecutive-failure budget + its flag, reconcile the `ERROR` exit code, ship the §17 fixture + canonical defect, de-overload `STALLED`, add `compare` to §5.1, harmonize the `phase`/FSM names and module names, state the token-budget unit, and define `edit_file` failure semantics), the document is a **Level 3** spec: minimal semantic inference, objectively testable conformance. A move to **Level 4 (verification-grade)** would additionally require a normative, exact-match exit-code/error-string catalogue and a pinned sandbox-creation protocol (F-016) — worthwhile but explicitly out of scope for this lab.

---

3. Findings Summary

16 findings: **0 CRITICAL · 4 HIGH · 8 MEDIUM · 4 LOW.** They cluster around three themes — *unpinned conventions that byte-identity depends on* (F-001, F-002, F-003, F-005), *overloaded / omitted or mis-referenced terms* (F-004, F-006, F-007, F-008, F-009, F-011, F-013), and *improvement/editorial*

clean-ups (F-012, F-014, F-015, F-016). No finding is a redesign; every P0/P1 fix is additive or a correction of a cross-reference.

ID	Sev	Location	Issue (one line)
F-001	HIGH	C-06, C-07, T-01, T-04, T-06, I-002	Iteration index base mixed 0-based (JSON) vs 1-based (prose) → byte-identity convention undefined.
F-002	HIGH	E-02, E-03, E-09, I-001, K-04	"N consecutive `ERROR`/`DENY` → terminate" points at K-04 (a *sandbox* bound) — threshold count + CLI flag undefined.
F-003	HIGH	C-08, K-02, §5.1, E-02, E-03	`ERROR` `final_outcome` exit code contradicted: "1 / 4" vs exit `4` reserved for `PULL_REQUIRED`.
F-004	MEDIUM	R-08, I-001, E-13, §3.2	`STALLED` overloaded (stuck-loop vs no-compact budget) with no threshold and no defined relationship.
F-005	HIGH	§0, §17, C-05, C-07, T-04, T-11	§17 fixture repo + canonical defect unspecified, yet T-04/T-11 assert concrete reproducible numbers.
F-006	MEDIUM	§3.3, E-05, R-16, R-18, §5.1	`compare` invoked in prose but absent from the §5.1 subcommand table.
F-007	MEDIUM	§5.2	GUI "read-only" attributed to I-007 (verifier-signal) — wrong invariant; no GUI invariant exists.
F-008	MEDIUM	§3.2, C-06	C-06 `phase` enum ≠ §3.2 FSM state names; "repair" is a phase but not a state (terminology drift).
F-009	MEDIUM	C-05, K-05, K-07	Token-budget unit undefined: K-07 uses *chars*, K-05 measures `| C_t |` with no char/token unit.
F-010	MEDIUM	C-03, §8	`edit_file` failure path (`old` not found → `applied=false`) has no loop/failure semantics.
F-011	MEDIUM	§1, R-15, I-009	Module name drift: actor names `loop.py`, I-009/R-15/traceability name `control_loop.py`; `sandbox.py` (traceability K-04) absent from actors.

ID	Sev	Location	Issue (one line)
F-012	LOW	§5.1, I-002	`--seed` "recorded in the banner" names no field; Ollama seed-determinism unguarded (acknowledged best-effort).
F-013	MEDIUM	C-06, R-09, §17	Instrumentation semantics unclear: does `search` increment `files_read`? No rule distinguishes read-tool vs search-tool.
F-014	LOW	C-06, I-013	`final_outcome` last-verdict win-rule not stated (which last verdict maps when the run ends `BUDGET_EXHAUSTED`).
F-015	LOW	§3.2, C-02	`NOOP` action feeds `STALLED`, but §3.2 FSM never shows a `NOOP`→`STALLED` transition.
F-016	LOW	K-04, K-02	Sandbox-creation protocol / cleanup timing unspecified; exit `4` for "un-writable" (E-10) collides with `PULL_REQUIRED 4`.

Severity discipline. No finding is CRITICAL: nothing makes the spec *impossible to implement or fundamentally unverifiable* — the deterministic core is solid and the probabilistic path is honestly bounded. The four HIGH items are *cross-consistency / convention defects that a verifier would trip on immediately*, not conceptual gaps. The MEDIUM set is the work that must be done before claiming conformance; the LOW set is improvement and MAY be deferred.

---

4. Detailed Findings

> Format per skill §5: ID · Severity · Location · Observation · Why it matters · Potential consequence · Recommended resolution. Findings are grouped HIGH → MEDIUM → LOW and ordered by ID.

F-001 — Iteration index base is not pinned (0- vs 1-based)

Severity: HIGH **Location:** C-06, C-07, T-01, T-04, T-06, I-002 **Observation.** The `trajectory.json`/`experiment.json` examples use **0-based** indices (`"iteration": 0`, and C-07 `"phases"` use `1/2/3` inconsistently), while the acceptance prose uses **1-based** counts: T-06 "stops at **iteration 5**", T-04 "repairs … on **iteration 3** … `iterations_to_verified == 3`", T-01 "one iteration …

`iterations_used == 1`". I-002 asserts byte-identity of the very arrays that carry these indices. **Why it matters.** The index base is a *convention*, not implementation freedom — it is serialized into the artifact and byte-compared by T-09/T-11. Two conforming implementations that disagree on the base produce **different bytes** for the same run, so I-002 cannot actually pass. **Potential consequence.** T-04 (`iterations_to_verified == 3`) and T-06 ("stop at iteration 5") are not writable without first inventing a base; byte-identity tests are non-deterministic in exactly the dimension they guard. **Recommended resolution.** Pin the convention in one sentence in C-06 and C-07: **1-based** `iteration` (`1 = first`), `iterations_used = max index`, and render C-06's example as `"iteration": 1`. Reconcile C-07 phases to the same base. Update T-04/T-06 prose to "the N-th iteration" so prose and JSON agree. *(P0)*

F-002 — "N consecutive `ERROR/DENY` → terminate" points at the wrong constraint and has no value/flag

Severity: HIGH **Location:** E-02, E-03, E-09, I-001, K-04 **Observation.** E-02 says "after **K-04** consecutive `ERROR` iterations the run terminates `ERROR`", but **K-04 is the *sandbox-size / subprocess-output* bound**, not an error budget. E-09 says "across **K** consecutive iterations … `DENIED_LOOP`" with `K` undefined. E-03 says "N consecutive `ERROR`". No constraint pins the threshold and none defines a CLI flag. **Why it matters.** A verifier cannot test "the run terminates after the k-th consecutive error" when k and its flag are unspecified. Worse, the cross-reference makes E-02 look *normative* when it actually inherits an unrelated sandbox constant. **Potential consequence.** Two implementers pick different tolerances (1 vs 5 vs "one more than `--max-iterations`") and produce different `final_outcome`/exit outcomes; the DENIED_LOOP / BUDGET_EXHAUSTED boundary is untestable. **Recommended resolution.** Introduce a **K-08** consecutive-failure budget with a default (e.g. `--max-consecutive-errors` default `2`, `--max-consecutive-denies` default `1`), and repoint E-02/E-03/E-09 to **K-08**, not K-04. State explicitly that a *single* `ERROR` feeds the next iteration (E-03) and only *K-08* consecutive `ERROR/DENY` terminates. *(P0)*

F-003 — `ERROR` `final_outcome` exit code is contradicted

Severity: HIGH **Location:** C-08, K-02, §5.1, E-02, E-03 **Observation.** C-08's closure table says `ERROR -> final_outcome ERROR -> exit 1 / 4`. But **K-02** and the §5.1 table both reserve **exit `4` exclusively for `PULL_REQUIRED`** (`run … 4 PULL_REQUIRED`), and map terminal non-`VERIFIED` outcomes to `1`. E-02/E-03 echo "exit `1/4`". So one of C-08 "1 / 4" and K-02 "1" is wrong. **Why it matters.** A CLI that returns `4` for an `ERROR` outcome collides with the `PULL_REQUIRED` channel, so an operator (and E-10's sandbox-unwritable `4`) cannot tell *why* `agent` exited `4`. The exit-code space is not a partition. **Potential consequence.** Tests over exit codes (T-03/T-12/E-10) cannot assert a code unambiguously; `ERROR` and `PULL_REQUIRED` indistinguishable to a shell caller.

Recommended resolution. Make the exit-code space a **partition**: `0` VERIFIED · `1` BUDGET_EXHAUSTED/STALLED/DENIED_LOOP/terminal-`ERROR` · `2` usage · `3` malformed-artifact load · `4` `PULL_REQUIRED` · and give `ERROR` from the *deterministic core* its own code (e.g. `5`), OR fold core-`ERROR` into `1` and **drop the "/ 4"** from C-08/E-02/E-03 and reassign E-10's unwritable-sandbox case to `5` so `4` stays `PULL_REQUIRED`-only. Pick one and make C-08/K-02/§5.1/E-10 agree. *(P0)*

F-004 — `STALLED` is overloaded and threshold-less

Severity: MEDIUM **Location:** R-08, I-001, E-13, §3.2 **Observation.** `STALLED` names **two distinct triggers**: (a) R-08/I-001 "repeated **identical** consecutive trajectories / no-op actions" (a stuck-loop, `K`/count undefined), and (b) E-13 "`--no-compact` under budget pressure" (a context-overflow). `STALLED` also appears in I-001's terminal set but §3.2's FSM never shows a transition into it. **Why it matters.** A verifier reading "terminate `STALLED`" cannot tell whether *identical-repetition* or *no-compact overflow* was asserted, nor the repetition threshold; the two are conflated under one label. **Potential consequence.** T-13 (no-compact → `STALLED`) and any stuck-loop test would assert on the same outcome string for different causes. **Recommended resolution.** Split into two outcomes or two labelled sub-codes — e.g. `STALLED:NOOP` (repetition, threshold pinned by a new/defaulted count, e.g. `--max-identical` default `2`) and `STALLED:BUDGET` (E-13 overflow) — and add the transition to §3.2's FSM. *(P1)*

F-005 — §17 fixture repository + canonical defect are unspecified, yet tests assert concrete numbers

Severity: HIGH **Location:** §0, §17, C-05, C-07, T-04, T-11 **Observation.** The chapter's central experiment names a *number* — `iterations_to_verified == 3` (T-04/T-11) — but the **fixture repository**, the **`parse_config` source**, the **canopy** defect ("wrong split delimiter"), and the **`VerifySpec`** command are all left to the implementer. "Detect on iteration 1, diagnose on 2, repair on 3" presupposes a specific fixture and a specific `MockPolicy` script, neither of which is pinned. **Why it matters.** This is the highest-leverage finding: the acceptance experiment is the chapter's thesis, and it is asserted *numerically* while its inputs are free. T-04/T-11 are not mechanically reproducible because the fixture is not part of the contract. **Potential consequence.** Two implementers ship different fixtures, different `MockPolicy` scripts, and different verifier commands; `iterations_to_verified` differs between them by design, defeating I-013 ("reproducible … not whatever the model did today"). **Recommended resolution.** Ship a **pinned fixture** as a versioned contract (e.g. `fixtures/parse-config/` with `src/config.py`, `test_config.py`, a one-line `tasks/parse-config.json` `Task` + `VerifySpec{cmd:"pytest -q"}`, and a `mock_policy.script` defining the 3-step detect/diagnose/repair `Action` sequence). Reference the fixture path in C-07/C-05 and T-04/T-11 so the numbers become testable. *(P0)*

F-006 — `compare` is invoked in prose but is not a subcommand

Severity: MEDIUM **Location:** §3.3, E-05, R-16, R-18, §5.1 **Observation.** §3.3 ("subsequent `compare`") and E-05 ("subsequent `compare`") and R-18/§5.1's `--baseline` "compare" all assume a `compare` operation, but the §5.1 subcommand table lists **only `run` / `experiment` / `inspect`** — no `compare`. The `--baseline` flag (R-18) is the de-facto carrier, but E-05 says "subsequent `compare`" as if it were a command. **Why it matters.** E-05 (a normative failure semantic) names an operation that does not exist in the interface catalog; an implementer has to invent where the comparison logic lives. **Potential consequence.** Inconsistency between the prose failure model and the CLI surface. **Recommended resolution.** Either add **`agent compare --baseline a --current b`** to §5.1 (emit a Δ report, exit `0`), and reword E-05 to "subsequent `compare`/`inspect`", or reword E-05/§3.3 to name `--baseline`. Pick one and keep E-05/§3.3/R-18 in lockstep. *(P1)*

F-007 — GUI "read-only" cross-references the wrong invariant

Severity: MEDIUM **Location:** §5.2 **Observation.** §5.2 says a read-only trajectory browser MAY be built "without running inference **(I-007)**", but **I-007 is the verifier-as-runtime-signal invariant**, not a GUI invariant. There is **no invariant** governing the MAY-GUI's read-only/offscreen behavior in this spec. **Why it matters.** A wrong cross-reference is worse than a missing one: a verifier following I-007 to justify the GUI constraint reads an unrelated invariant and a real GUI-readiness claim is *unspecified* even though the spec gestures at one. **Potential consequence.** GUI behavior (if built) is untested; the wrong reference misleads. **Recommended resolution.** Point §5.2 at the correct invariant — or, since the GUI is `MAY`, drop the parenthetical and state the GUI-read-only property ("never runs inference or re-touches the sandbox; I-006 carried for the artifact readers") without a dangling `I-007`. *(P1)*

F-008 — C-06 `phase` enum and §3.2 FSM state names do not align

Severity: MEDIUM **Location:** §3.2, C-06 **Observation.** §3.2's FSM states are `OBSERVE/REASON/PERMIT/ACT/VERIFY/FEEDBACK` (plus terminal set), while C-06's per-row `phase` enum is `observe|inspect|search|propose|modify|verify|repair|stop`. The two vocabularies differ (`inspect/search/propose/modify/repair/stop` vs `OBSERVE/REASON/ACT/FEEDBACK`), and `repair` is a *phase* but has no corresponding FSM state. **Why it matters.** The instrumented `phase` is a serialized field (T-07 asserts the full field set). If its enum is not reconciled with the FSM, the `phase` value is unvalidated and two implementers will disagree on its value space. **Potential consequence.** `phase` cannot be schema-validated coherently; T-07's "full field set" check under-specifies the enum. **Recommended resolution.** Either (a) map each `phase` to the FSM `state` it belongs to (a small mapping table) and validate `phase \in {…}` in `schemas/trajectory.json`, or (b) rename to a single shared vocabulary. Add `NOOP`/`stop` transitions for F-015. *(P1)*

F-009 — Token-budget unit (chars vs tokens) is undefined

Severity: MEDIUM **Location:** C-05, K-05, K-07 **Observation.** K-07 defines the *surrogate* `tokens.estimated` via a **character** count ("`len(C_t chars)`"), while K-05 fires compaction when "`| C_t |` > BUDGET" with no unit. A token budget and a character count are different magnitudes; the compaction trigger's unit is therefore undetermined. **Why it matters.** K-05 is a constraint and is asserted by T-13; its trigger magnitude being a char- vs token-miscount changes when compaction fires. **Potential consequence.** T-13 (compaction triggered / not triggered) is sensitive to an unstated unit choice. **Recommended resolution.** State the unit explicitly — K-05 in **characters** (consistent with K-07's surrogate), with a named BUDGET default, and reword `| C_t |` to "**|C_t| (chars)**". Keep K-07 consistent. *(P1)*

F-010 — `edit_file` failure (`applied=false`) has no loop semantics

Severity: MEDIUM **Location:** C-03, §8 **Observation.** C-03's `edit_file` returns `EditResult{applied: bool, diff: str}` — i.e. a failed edit (`old` not found → `applied=false`) is possible — but §8 defines no edge case for a failed edit, and §3.2's FSM never handles "edit applied=false on the next iteration". E-02 covers a policy emitting an *unrecognizable action*; a *recognized but failed* edit is a gap. **Why it matters.** The verify-feedback loop's whole point is recovery from a failed modification; the spec says nothing happens on `applied=false`, so the next observation is undefined. **Potential consequence.** Two implementers differ (treat as no-op and continue vs treat as an `ERROR` that counts toward K-08). **Recommended resolution.** Add an **E-14** "edit applied=false": the failed `EditResult` is surfaced into the next `observation` verbatim (so the repair policy can correct its `old`/`new`); a failed edit does *not* silently advance, and repeated failed edits feed the K-08 budget. Cross-link from C-03 and §3.2. *(P1)*

F-011 — Module-name drift (`loop.py` vs `control_loop.py`); `sandbox.py` unowned

Severity: MEDIUM **Location:** §1, R-15, I-009, K-04, §11 traceability **Observation.** The `ControlLoop` actor is named `loop.py` (§1), but I-009, R-15, and the §11 traceability rows name the same module `control_loop.py`. Separately, K-04's traceability row points at `sandbox.py`, but **no actor or contract owns `sandbox.py`** (sandbox creation is described only in prose). **Why it matters.** The "source/graph scan" T-02/I-009 names a precise import boundary; the boundary list (`control_loop.py` vs `loop.py`) is the thing being asserted. If the module list is inconsistent, the scan's allow/deny set is ambiguous. `sandbox.py` having no owner hides a real module from the core-import discipline. **Potential consequence.** I-009's "MUST NOT import a network client" is asserted over an inconsistent module list; `sandbox.py` (a real module) is neither in the actor model nor in the scan boundary. **Recommended resolution.** Standardize on `control_loop.py` everywhere (§1 actor + §0 prose + traceability), and

add `sandbox.py` to the I-009/R-15 core module list and to §1 (as an actor or "module" row). *(P1)*

F-013 — Instrumentation semantics: does `search` count as a `files_read`?

Severity: MEDIUM **Location:** C-06, R-09, §17 **Observation.** §17's fields distinguish `tool_calls` from `files_read`/`files_modified`, but the spec never states whether a **`search`** or **`list_files`** call increments `files_read` (they *discover* files without reading them), or whether `tests_executed` counts a verifier `run_shell` that *didn't execute any test*. **Why it matters.** `files_read`, `files_modified`, `tests_executed` are *metrics over the trajectory* and are byte-compared (I-002); their *counting rules* must be pinned or two implementers produce different numbers for the same run. **Potential consequence.** Byte-identity (I-002/T-09) over the very fields §17 demands, defeated by an unstated counting rule. **Recommended resolution.** Add a one-paragraph **counting rules** note under C-06: `files_read` = distinct paths opened by `read_file` (not `search`/`list_files`); `files_modified` = distinct paths with a successful `edit_file` (`applied=true`); `tests_executed` = count of `run_shell` verdicts in the verifier (0 when the verifier's command exits the *runner*, not a test); `tool_calls` = all calls incl. denied. *(P1)*

F-012 — `--seed` "recorded in the banner" names no field; Ollama seed-determinism unguarded

Severity: LOW **Location:** §5.1, I-002 **Observation.** §5.1 says `--seed N` is "recorded in the banner", but the only banner field in C-06 is `availability_banner`, which is *about model availability*, not `seed`. I-002 (byte-identity) explicitly **excludes** the Ollama path, so a seeded-but-different real run is unguaranteed — which is fine, but the "recorded in the banner" claim is unattached. **Why it matters.** Minor: a real-run's provenance (which seed produced this trajectory) has no field. **Potential consequence.** No deterministic guarantee, and the seed has nowhere to live in `trajectory.json`. **Recommended resolution.** Either add a `seed: int | null` field to C-06 (top-level, null on mock), or reword §5.1 to "recorded in `--verbose` logs", not the banner. *(P2)*

F-014 — `final_outcome` last-verdict win-rule not stated

Severity: LOW **Location:** C-06, C-08, I-013 **Observation.** C-08 gives a closure from *verdict class* to *outcome*, but says nothing about which verdict "wins" when the run ends on `BUDGET_EXHAUSTED` — i.e. is the last verdict's `FAILED`/`ERROR` recorded, or only the final `BUDGET_EXHAUSTED`? **Why it matters.** `final_outcome` is a single scalar in C-06; the rule that picks it from a multi-iteration run's history is unstated, so a verifier cannot assert which value T-05/T-06 must see. **Potential consequence.** Two implementers could emit different `final_outcome` strings for the same BUDGET_EXHAUSTED

run. **Recommended resolution.** Add to C-08: "**`final_outcome` is always the terminal-stopping outcome, not the last verdict** — on BUDGET_EXHAUSTED/STALLED/DENIED_LOOP it is that label; `ERROR` outcome is only reached by the terminal-error path (E-02/E-03/K-08)." *(P2)*

F-015 — `NOOP` action has no FSM transition into `STALLED`

Severity: LOW **Location:** §3.2, C-02 **Observation.** C-02 says `NOOP(note)` "feeds STALLED detection (R-08)", but §3.2's FSM never shows an `NOOP` action routing into a `STALLED` terminal — the diagram goes `ACT → VERIFY → (VERIFIED | next OBSERVE)`, with no "policy emitted NOOP" branch. **Why it matters.** A reader simulating the loop has no defined transition from "policy returns `NOOP`" to "run terminates `STALLED`". **Potential consequence.** The stuck-loop branch of R-08/I-001 is a described-but-not-shown transition. **Recommended resolution.** Add a NOOP branch to §3.2's diagram: `Policy → NOOP · (consecutive count >= K-08-stalled?) → STALLED`, consistent with F-004's split. *(P2)*

F-016 — Sandbox-creation protocol and exit-code collision

Severity: LOW **Location:** K-04, K-02, E-10 **Observation.** K-04 bounds the *content* of the sandbox but not its *creation* (copy protocol: shallow vs `shutil.copytree`, symlink handling, ignore-list), and E-10's "sandbox root un-writable → exit `4`" collides with `PULL_REQUIRED`'s `4` (a consequence of F-003). Once F-003 is resolved this one is mostly a one-line cleanup, but the copy protocol itself is a real (minor) gap. **Why it matters.** A sandbox that `shutil.copytree`s symlinks out of the bootcamp repo is a leak; the spec says "copy" but not "how". **Potential consequence.** Two implementers may copy symlinks out of tree, defeating I-003. **Recommended resolution.** Add one sentence to K-04: "sandbox copy is 'shutil.copytree(symlinks=False, **ignore=**) — symlinks are not followed and are not copied as links; the sandbox is removed on the run's finally-block regardless of outcome." *(P2)*

---

5. Requirements Review

Requirements R-01…R-18 are **observable** and, with the exception of the cross-reference defects, precise. Each names a condition, an input, an obligation, and (for the normative ones) a consequence, so "what must happen, when, and if the condition cannot hold" is mostly answerable. The requirements correctly keep the deterministic core and the probabilistic path in one document while honestly bounding the latter (R-13/R-14).

Two requirements are aspirational rather than observable: **R-18** (System-A vs System-B) is explicitly `SHOULD`/non-gated — acceptable, it is the one requirement the spec *chose* to leave open. **R-04** ("tool execution extends the model

into the environment") reads as a thesis dressed as a requirement; its observable content is fine but the sentence is rhetorical. No requirement conflicts *with another*; the conflicts found (F-002/F-003) are in the contract/constraint layer, not the requirement layer. No material requirement appears to be missing except an explicit **out-of-scope** statement (subagents §10 — see F-004's note and §17).

6. Interface and Data-Contract Review

The contract layer is the strongest part of the spec: **C-02** (`Policy` protocol + closed `Action` tag-union), **C-03** (pinned `TOOL_SET` with in/out schemas), **C-04** (permission `ALLOW/DENY` with a defined precedence), and **C-05** (`VerifySpec`/`Verdict`) are clean, closed, and testable. **C-06**/`C-07` (the artifacts) pin a versioned schema and field set — the right move for byte-identity.

The defects here are *convention-level*: the iteration index base (F-001), the per-field counting rules (F-013), and the `phase` enum's relation to the FSM (F-008) all live in C-06 and must be reconciled before C-06 is a true *contract* rather than a *template*. **C-03** must add the `applied=false` branch (F-010). Serialization is otherwise well-handled: JSON artifacts with a version field and a load-time schema gate (I-012). No schema is internally incoherent; the issues are *completeness of the value spaces*.

7. State and Failure Review

§3.2 gives a clear FSM (`RECEIVED→OBSERVING→REASONING→PERMITTING→ACTING→VERIFYING→FEEDBACK` with terminal set) and C-08 a verdict/outcome/exit closure — a genuine strength for a lab spec. The failure model is mostly complete (E-01…E-13, plus the to-be-added E-14 for F-010).

The gaps are at the *transition* level, not the *state* level: (1) the **consecutive-error / consecutive-deny** terminations reference the wrong constraint and have no threshold (F-002); (2) `STALLED` is overloaded and has no FSM transition (F-004, F-015); (3) `edit_file` **applied=false** has no next-step (F-010). Retry behavior is *deliberately absent* (the spec prefers "let the policy see the result and continue" over "retry an operation"), which is a coherent design choice and should be stated as such rather than left implicit. Cancellation/timeout are partially covered by K-04 (subprocess cap) but E-10/K-04 need the copy-protocol sentence (F-016). Partial completion (a `VERIFIED` reached *then* a later iteration) is implicitly ruled out by I-006's "only path to VERIFIED" — good, but should be stated.

8. Determinism and Algorithm Review

Determinism is the spec's central claim and is handled well *in intent*: I-002 (byte-identity of the mock path), K-06 (zero-network), K-07 (deterministic sur-

rogate fields with a named formula). The defect is that **the conventions the determinism depends on are themselves unpinned**: the iteration base (F-001), the per-field counting rules (F-013), and the token-budget unit (F-009). A byte-identity invariant is only as strong as the conventions that *define the bytes* — and those three are exactly where v0.1 is loose. The surrogate formula in K-07 is illustrative ("e.g.") rather than normative; for I-002 to be *testable by exact bytes* the formula must be **pinned, not exemplified** (P1 part of F-009's resolution). No rounding/tie-breaking issue arises because the only math is the additive surrogate.

9. Edge-Case Review

E-01…E-13 is a broad, well-formed catalog (missing input, malformed input, unavailable dependency, conflicting input, resource exhaustion analog via K-04) — above what most lab specs provide. Each edge names a deterministic outcome (an exit code and/or a `final_outcome`).

After integration, the catalog is near-complete except: **E-14** (edit `applied=false`, F-010) and the **consecutive-error/deny** terminations folded into K-08 (F-002). The one *collision* is E-10's exit `4` vs `PULL_REQUIRED`'s `4` (F-016/F-003) — resolve by making the exit space a partition (F-003). E-01 (empty task) and E-07 (`--max-iterations 0`) correctly pre-allocate-check before sandbox creation (good ordering discipline).

10. Non-Functional Requirement Review

The spec's non-functional posture is sound: performance is *deliberately not* a numeric gate (K-04 bounds the subprocess but sets no wall-clock target), which matches the "understand the loop, not ship a product" framing. Determinism/reproducibility (I-002, K-07, F-001/F-009/F-013) is the real non-functional focus and is mostly measurable once the three conventions are pinned. Security is addressed (permission layer, sandbox isolation, LLM-free core). Observability is the strongest NFR (the whole §17 instrumentation). No unmeasurable "must be fast"-style requirement is present.

11. Security and Trust-Boundary Review

This is a **strength** and the chapter's key thesis made mechanical: the model's authority is bounded *outside itself* (I-008 "permission precedes execution", R-05 "permissions enforced outside the model", tested by T-03 with a `../escape` adversarial). Sandbox isolation (I-003, dual-enforced in tools + permissions) and the non-recursive, non-self sandbox (I-011, E-08/T-10) close the "agent editing its own test harness" vector. The LLM-free-core discipline (I-009, R-15, K-06, T-02) is a well-reasoned separation of the trust boundary.

The one trust-boundary gap is at the *sandbox boundary itself*: the copy protocol (F-016) — an unfollowed-symlink copy — is the one place a malicious/insecure

fixture could escape I-003, and it should be pinned. No secret-handling or authentication concern arises (single-principal, local daemon only). No privilege escalation is in scope.

12. Observability and Provenance Review

Observability is the central deliverable and is well-specified: the full §17 field set per iteration (C-06), the versioned artifacts, and the model-availability banner (R-14/E-11/E-12) mean *what happened and why* is largely recoverable. After the review the provenance picture is complete except: the `--seed` value has no field (F-012) and the per-field counting rules (F-013) make the *metrics* provenance-stable. No run identifier/timestamp is captured, which is acceptable for an offline, deterministic lab artifact (the artifact is self-describing via `trajectory_version` + inputs), but a run-id would be an easy Level-4 addition (deferred).

13. Testing and Verification Review

The test catalogue T-01…T-13 is strong and, on the whole, *maps one-to-one to invariants* — every invariant has a test and every test cites an invariant (I-001→T-06, I-002→T-09, I-003→T-03/T-10, I-006→T-01/T-05, I-009→T-02, I-013→T-11). This is exemplary for a lab spec.

The tests that are **not yet mechanically writable** from v0.1 are exactly the HIGH findings: **T-04** (needs the F-005 fixture + the F-001 index base) and **T-06**/T-05 (need the F-002 consecutive-failure budget + F-001 base). **T-09/T-11** (byte-identity) depend on F-001/F-009/F-013. Once those are pinned, the catalogue is genuinely conformance-testable. No *two-tester-disagreement* ambiguity remains after P0/P1. T-02 ("no socket in deterministic core") is a source-scan, not a unit test — correctly framed as advisory.

14. Metrics and Evaluation Review

The §17 metrics (`iterations_used`, `iterations_to_verified`, `tokens`, `files_read`, `files_modified`, `tests_executed`, `final_outcome`) are well-chosen and each has a denominator/population *except* the ones whose **counting rule is unpinned** (F-013: what increments `files_read`/`files_modified`/`tests_executed`; what is `iterations_to_verified`'s population). `iterations_to_verified` is a *count-to-VERIFIED* metric whose population (which run, which fixture) is undefined pre-F-005 — so it is not yet "independently reproducible from specified evidence." After F-005 + F-013, every metric has a pinned population, definition, and (for the mock path) a deterministic formula. No metric is "supplied by the component being evaluated" in a way that defeats conformance — the verifier's verdict is the agent's *own* output, which is the intended design.

15. Traceability Review

§11's `id → module → test` matrix is a strength: it closes the intent→requirement→contract→invariant→test→ chain for every ID, and (now that R-10 is present) has no dangling gap. The traceability is *mostly* coherent; the defects it surfaces are that a few *module names* in the "where realized" column are inconsistent with the contract/actor model: **`control_loop.py vs loop.py`** (F-011, in the traceability rows themselves) and **`sandbox.py`** (present in the K-04 row, absent from actors/I-009). Fixing F-011 makes the traceability column a true contract. The chain is otherwise unbroken; no test is orphaned (F-016's new E-14 / C-03 change would extend, not break, it).

16. Internal-Consistency Review

The most consequential findings are consistency defects *inside* the document (F-002 cross-references the wrong constraint; F-003 contradicts the exit-code partition; F-011 module names; F-007 a wrong invariant reference; F-008 two vocabularies for one concept). Each is a one-to-three-line reconciliation, not a redesign. After P0/P1 the document is internally consistent: one iteration base, one exit-code partition, one module-name per component, one `phase` vocabulary, and `STALLED` split into its two sub-codes. No *later-section-implicitly-overrides-earlier* conflict remains after the P0 fixes.

17. Architecture Review

The §15/§0 architecture is correct and *supports* the requirements: policy→permissions→tools→verifier→feedback with the LLM confined to `policy.py` is the right decomposition for the chapter's thesis. Dependency direction is healthy: the deterministic core depends on no network; `policy.py` is the only egress; artifacts flow *down* through `report.py`; the model never flows *up* (I-006/I-009). No *state ownership* is ambiguous except the sandbox lifecycle (F-016's copy protocol). **No architectural change is recommended.** The only structural change worth making is adding `compare` to the CLI surface (F-006) and pinning `sandbox.py` as a module (F-011) — both additive, both non-redesigning.

18. Implementation-Agent Readiness

> *Could a strong coding agent implement this specification without asking material semantic questions?*

YES — WITH MINOR CLARIFICATIONS (i.e. *Level 2+ today; Level 3 after P0/P1*).

A strong agent can build the deterministic core *today* with minimal inference. But it would necessarily *guess* — and two agents would guess *differently* — exactly the material questions below. These are the blocking semantic questions; resolving them is the entire P0/P1 set.

Minimum blocking questions (P0/P1):

1. *(F-001)* Is the iteration index 0- or 1-based? — affects every byte-comparison.
2. *(F-002)* What is the consecutive-`ERROR`/`DENY` termination threshold, and what CLI flag carries it?
3. *(F-003)* Which exit code does a deterministic-core `ERROR` outcome use, vs `PULL_REQUIRED`'s 4?
4. *(F-005)* Which fixture repository, `parse_config` source, and canonical defect do T-04/T-11 test against?
5. *(F-013)* What increments `files_read`/`files_modified`/`tests_executed`?
6. *(F-009)* Is the token/compaction budget measured in characters or tokens, and what is its default?
7. *(F-010)* When `edit_file` returns `applied=false`, what happens on the next iteration?
8. *(F-004)* Are the two `STALLED` causes one outcome or two, and what is the repetition threshold?
9. *(F-006)* Is `compare` a subcommand or a `--baseline` flag?
10. *(F-011)* Is the control-loop module `loop.py` or `control_loop.py`, and does the I-009 core include `sandbox.py`?

The **P2** questions (F-012 seed field, F-014 win-rule, F-015 NOOP transition, F-016 copy protocol + exit collision) are non-blocking improvement — reasonable defaults exist. **Readiness gate:** the document is *READY* once P0/P1 (F-001–F-005, F-006–F-011, F-013) are integrated; P2 may follow.

---

19. Quality Scorecard

#	Dimension	Score	Note
1	Scope clarity	4	Clear deterministic/probabilistic split; only the out-of-scope statement (subagents §10) is missing.

#	Dimension	Score	Note
2	Terminology	3	Strong, but `STALLED` (F-004) and the `phase`/FSM mismatch (F-008) over-load two terms.
3	Requirement precision	4	R-01…R-18 mostly observable; R-04 rhetorical, R-18 intentionally open.
4	Interface completeness	4	C-02…C-05 clean; C-06/C-07 complete once counting/base/`phase` are pinned; C-03 needs the `applied=false` branch.
5	Data-contract completeness	4	Versioned schema + load gate; value-space conventions (F-001/F-008/F-009/F-013) to pin.
6	State/lifecycle definition	4	Clear FSM + C-08 closure; consecutive-failure/`STALLED` transitions (F-002/F-004/F-015) to add.

#	Dimension	Score	Note
7	Algorithm precision	3	Loop/closure precise; byte-identity weakened by 3 unpinned conventions + an "e.g." surrogate formula.
8	Failure semantics	4	E-01…E-13 broad; F-002 mis-reference + E-14 (F-010) + exit partition (F-003) to reconcile.
9	Edge-case coverage	4	Above average; one collision (E-10 exit 4) and two additions (F-010, consecutive-failure).
10	Non-functional requirements	4	Sound; determinism focus; no unmeasurable claims.
11	Security specification	4	Permission-outside-model + sandbox + LLM-free core (chapter thesis, made mechanical); only the copy protocol (F-016) open.

#	Dimension	Score	Note
12	Observability/provenance	4	Full §17 instrumentation; `--seed` field + counting rules (F-012/F-013) to add.
13	Testability	4	Invariant<->test pairing exemplary; T-04/T-06 not yet writable until P0 (F-005/F-001/F-002).
14	Evaluation/metrics	3	Good metric set; populations/counting rules (F-013) + fixture (F-005) to pin.
15	Traceability	4	Complete §11 matrix; module-name drift (F-011) to harmonize.
16	Internal consistency	3	6 in-document consistency defects (F-002/F-003/F-007/F-008/F-009/F-011), all one-to-three-line reconciliations.
17	Architecture consistency	4	§15 architecture supports the requirements; no redesign needed.

#	Dimension	Score	Note
18	Implementation readiness	3	YES-with-minor-clarifications today; Level 3 after P0/P1.

Mean ≈ 3.6 / 5 — an *implementable-to-strong* spec whose gap to Level 3 is concentrated in *convention pinning and internal consistency*, not in missing behavior or a weak architecture.

---

20. Remediation Plan

P0 — Blocking (must resolve before implementation / before claiming conformance)

- **F-001** Pin the iteration index base (recommend **1-based**), reconcile C-06/C-07 examples and T-01/T-04/T-06 prose. *(touches C-06, C-07, T-01, T-04, T-06, I-002)*
- **F-002** Add **K-08** consecutive-`ERROR`/`DENY` budget with a default + CLI flag; repoint E-02/E-03/E-09 from K-04 to K-08.
- **F-003** Make the exit-code space a **partition** (give terminal core-`ERROR` its own code, fold `PULL_REQUIRED` into 4-only; reassign E-10's unwritable sandbox out of 4); harmonize C-08/K-02/§5.1/E-02/E-03/E-10.
- **F-005** Ship a **pinned §17 fixture** (repo, `parse_config` source, canonical defect, `VerifySpec`, `MockPolicy` script); reference it from C-07/C-05 and T-04/T-11.

P1 — Important (resolve before claiming conformance)

- **F-004** Split/label `STALLED` (e.g. `STALLED:NOOP` vs `STALLED:BUDGET`), pin the repetition threshold; add to §3.2.
- **F-006** Add `agent compare` to §5.1 (reword E-05/§3.3) or reword prose to `--baseline`.
- **F-007** Fix §5.2's invariant reference (drop the wrong `I-007`; state the GUI read-only property plainly).
- **F-008** Reconcile C-06 `phase` enum with §3.2 state names; schema-validate `phase`.
- **F-009** State the token/compaction budget **unit (chars)** + default; make the K-07 surrogate formula **normative** (drop "e.g.").
- **F-010** Add **E-14** "edit `applied=false`" loop semantics; cross-link C-03/§3.2.
- **F-011** Standardize on `control_loop.py`; add `sandbox.py` to the I-009/R-15 core list and to §1.

- **F-013** Add **per-field counting rules** under C-06 (`files_read/files_modified/tests_executed/tool`
- (Out-of-scope statement — fold into §0: *single-agent, single-iteration-per-action; subagents §10 explicitly out of scope.*)

P2 — Improvement (MAY defer; per the skill, deferral is acceptable)

- **F-012** Add a top-level `seed: int | null` field to C-06, or reword §5.1.
- **F-014** State the `final_outcome` win-rule in C-08.
- **F-015** Add the `NOOP → STALLED` transition to §3.2.
- **F-016** Pin the sandbox copy protocol (`copytree(symlinks=False, …)`, finally-block cleanup); this also closes F-003's exit collision for E-10.

Integration discipline (per the `spec-writing` uplift campaign): apply P0 first, then P1, then bump the header `v0.1 → v0.2 (P0/P1 … integrated; P2 deferred)`, commit as `review(ch15):` / `fix(ch15):`, and record F-IDs inline where each fix lands.

---

21. Final Verdict

```
Specification maturity:
Level 3 - Implementation-grade, PENDING the P0/P1 set (presently a strong Level 2+).

Implementation readiness:
READY WITH MINOR FIXES - a strong coding agent can build the deterministic core today,
but the P0/P1 conventions must be pinned so two implementers (and a verifier) agree on
byte-level behavior. P2 items (F-012…F-016) may be deferred.

Primary blocker:
The §17 fixture + canonical defect (F-005) and the three unpinned conventions
it depends on (iteration base F-001, consecutive-failure budget F-002, ERROR exit code F-003
leave the chapter's central acceptance experiment non-deterministic.

Most important improvement:
Pin the §17 fixture and the iteration/edge conventions - that single cluster of additive,
non-redesigning edits carries the spec from "Level 2 with ambiguities" to a genuinely
conformance-testable Level 3.
```

Recommendation. Apply P0 then P1, bump to v0.2, and — per the optional review-and-uplift workflow — leave P2 deferred or fold F-016 opportunistically (it also closes F-003). After that the document is the strongest artifact of its kind in this bootcamp's labs track: a closed-loop agent spec whose every claim is either mechanically testable or honestly bounded.